DocuSense

Learning, demo, and interview guide

This guide helps you explain the implementation accurately after studying and running it. Use the resume bullets only once you can reproduce the behavior, interpret the experiments, and explain your own changes. Do not imply that you trained MiniLM or Qwen, or invented their architectures.

1. A thirty-second explanation

I built a local document intelligence prototype that converts PDFs into traceable text passages, retrieves relevant evidence with pretrained embeddings, and optionally generates cited answers through a local language model. I also implemented corpus analytics, exploratory embedding clustering, and a reproducible evaluation harness comparing semantic retrieval with a TF-IDF baseline.

2. What this demonstrates

Data science: inspecting extracted data quality, descriptive statistics, keyword distributions, PCA, clustering, relevance annotation, metric design, ablations, and error analysis. Explain why a small development dataset cannot justify broad performance claims.

ML engineering: packaging pretrained inference, deterministic preprocessing, model lifecycle and local caching, stable identifiers, input validation, latency measurement, failure handling, dependency capture, and automated tests. Clustering is fitted ML; the encoder and generator are integrated pretrained models.

AI engineering: retrieving evidence, designing constrained prompts, local inference, explicit abstention, source ID validation, and documenting the difference between provenance and factual support. RAG is one component of the broader engineering project.

Software engineering: typed modules, UI/core separation, VS Code debugging and tasks, CI, safe file handling, and reproducible demo assets. Avoid claiming service-level reliability, scalability, or production security that was not measured.

3. A five-minute demonstration

Step 1: Start Streamlit and load the original demo corpus. Show real counts and the active retrieval engine. Explain that demo documents are original educational content and their labels are development examples.

Step 2: Search for how data drift differs from concept drift. Open the retrieved passage, page reference, score, and stable chunk ID. Explain why a similarity of 0.7 is not a seventy-percent chance of correctness.

Step 3: If Ollama is running with the correct model, request a local RAG answer. Inspect the cited passages. If generation is unavailable, show the explicit failure message and evidence-only behavior. Do not conceal this distinction.

Step 4: Open analytics. Describe the word distribution and explain how original page counts avoid inflation from overlapping windows. Run K-Means on semantic embeddings, then explain why PCA is a visualization and why clusters are not supervised classes.

Step 5: Load the reviewed demo labels and run retrieval evaluation. Explain one query-level failure, Recall@1 versus Recall@5, and why MRR rewards finding a relevant passage earlier. Open reports/ablation.csv to compare design choices using actual measurements.

4. Questions you should be ready to answer

Why RAG instead of fine-tuning? Retrieval provides access to a changing document corpus and source passages without changing generator weights. It can still retrieve irrelevant evidence or generate unsupported claims. Fine-tuning may adapt style or domain behavior but does not replace provenance and retrieval evaluation.

Why use a TF-IDF baseline? It is inexpensive, interpretable, and often strong for exact terminology. A baseline reveals whether pretrained dense representations actually help on the measured task. A negative or mixed result is valid evidence, not a reason to hide the experiment.

Why normalize embeddings? Dividing by vector length removes magnitude from cosine ranking. The normalized dot product computes cosine similarity efficiently across all passages using one matrix multiplication. Normalization does not make the score a probability.

What does overlap cost? It preserves boundary context but increases index size, duplicate evidence, encoding work, and the chance that multiple top results come from one page. Page-level recall deduplicates relevant hits in the numerator but does not remove their ranks.

How do you prevent leakage in evaluation? Freeze a final set of questions and labels before selecting settings, use additional documents, and report development and held-out results separately. The included demo set is explicitly for development and demonstration, so it is not presented as a final test benchmark.

Why MRR@5 rather than full MRR? The application retrieves five passages for evaluation. Its reciprocal rank is therefore truncated at depth five. Naming the cutoff avoids implying that the entire corpus ranking was inspected.

Are citations enough to stop hallucinations? No. They validate source membership only. A generated claim can cite a real passage that does not support it. Human claim-support review or a separately validated entailment evaluation is needed.

Why does a PDF need OCR? Scanned pages can contain images of text without a machine-readable text layer. The parser cannot infer that text from image pixels. The application reports the limitation rather than silently indexing an empty document.

What does silhouette measure? Relative cohesion and separation in a chosen distance space. It does not establish that the groups correspond to meaningful classes. The implementation uses cosine silhouette on a bounded stratified sample and labels the result accordingly.

What would you change for scale? Profile encoding, retrieval, and generation separately, then test an approximate nearest-neighbor index and batching. Keep corpus and model versions with index artifacts. Add tenant isolation and operational controls before exposing the app remotely.

5. Read the code in this order

Start with pdf_parser.py and chunking.py to understand source provenance. Read embeddings.py and retrieval.py for model integration and cosine ranking. Use the retrieval tests to verify a hand-computable vector example. Then read evaluation.py and calculate a sample result manually.

Read rag.py next, tracing the unavailable-server path and invalid-citation path. Continue to analytics.py for exploratory ML. Finally read app.py to see how UI state coordinates the modules. Change one chunk setting and inspect which IDs and measurements change.

6. Practice experiment

Run scripts/benchmark.py --semantic. Read every row in reports/ablation.csv and inspect at least three failed queries in the query CSV files. Record a hypothesis, such as larger windows reducing answer granularity. Test it using additional questions and keep the final evaluation independent.

Measure indexing separately from query latency. Cold model loading and warm embedding caches affect timings. If comparing machines, record CPU, memory, package versions, model version, corpus hashes, and run order. Do not claim a speedup from a single uncontrolled observation.

Write a short error-analysis note that separates extraction failures, retrieval misses, missing labels, and generation mistakes. These are different failure modes and require different fixes.

7. Resume language

Built DocuSense, a local PDF intelligence application with pretrained semantic retrieval, traceable evidence passages, Ollama-based RAG, and validated source references using Python, Streamlit, Sentence Transformers, and NumPy.

Implemented a reproducible retrieval evaluation pipeline with manually reviewed page labels, Recall@1/3/5, MRR@5, query-level exports, and TF-IDF versus dense-retrieval ablations across chunking configurations.

Developed exploratory corpus analytics with K-Means, PCA, keyword summaries, and valid cosine silhouette scoring; added deterministic tests, VS Code configurations, and GitHub Actions CI.

Only add numeric achievements after reproducing measured results on a clearly named dataset. Prefer phrasing such as evaluated on eighteen demo questions over an unsupported claim such as achieved industry-leading accuracy. Never invent a percentage improvement or business impact.

8. Honest ownership statement

This project was developed with AI coding assistance. My responsibility is to understand, validate, and improve its design. In an interview, I can explain the data flow, reproduce the experiments, identify limitations, and make a change live. I used pretrained models; I did not train a new language model.

9. Personal extension checklist

Add ten independently written questions from unfamiliar documents. Inspect all their relevant pages. Keep these separate from the demo development set. Analyze errors before adding a hybrid retriever or reranker. Update the report with actual measurements and explain one design decision you changed based on evidence.

Add a supervised document-type classifier only if you collect enough reliable labels and can evaluate it without leakage. More model names do not automatically strengthen the project. A defensible experiment and a well-understood limitation are often more valuable.